Database Systems Evolution

UA.DETI.CBD
José Luis Oliveira / Carlos Costa

Outline

- Why do we need storage system
- How they evolved along the time
- Milestone solutions
- Current landscape

Thinking about Data Systems

- Many applications today are **data-intensive**, as opposed to **compute-intensive**.
  - AI-driven workloads reintroduced compute-intensive paradigms
- Raw CPU power is rarely a limiting factor for these applications
  - bigger problems are usually the **amount** of data, the **complexity** of data, and the **speed** at which it is changing.

AI and Distributed Data System

- Data Sharding
  - Massive datasets must be partitioned across nodes to enable parallel training and scalable inference.
- Distributed Storage
  - Petabyte-scale datasets and model artifacts require scalable, fault-tolerant storage architectures.
- Replication
  - High availability, geo-distribution, and fault tolerance remain mandatory for AI production systems.
- Consistency Trade-offs
  - Different stages (training, feature stores, inference pipelines) tolerate different consistency levels.
- Vector Search Infrastructure
  - Embedding-based retrieval (RAG, semantic search) depends on distributed, partitioned vector indexes.

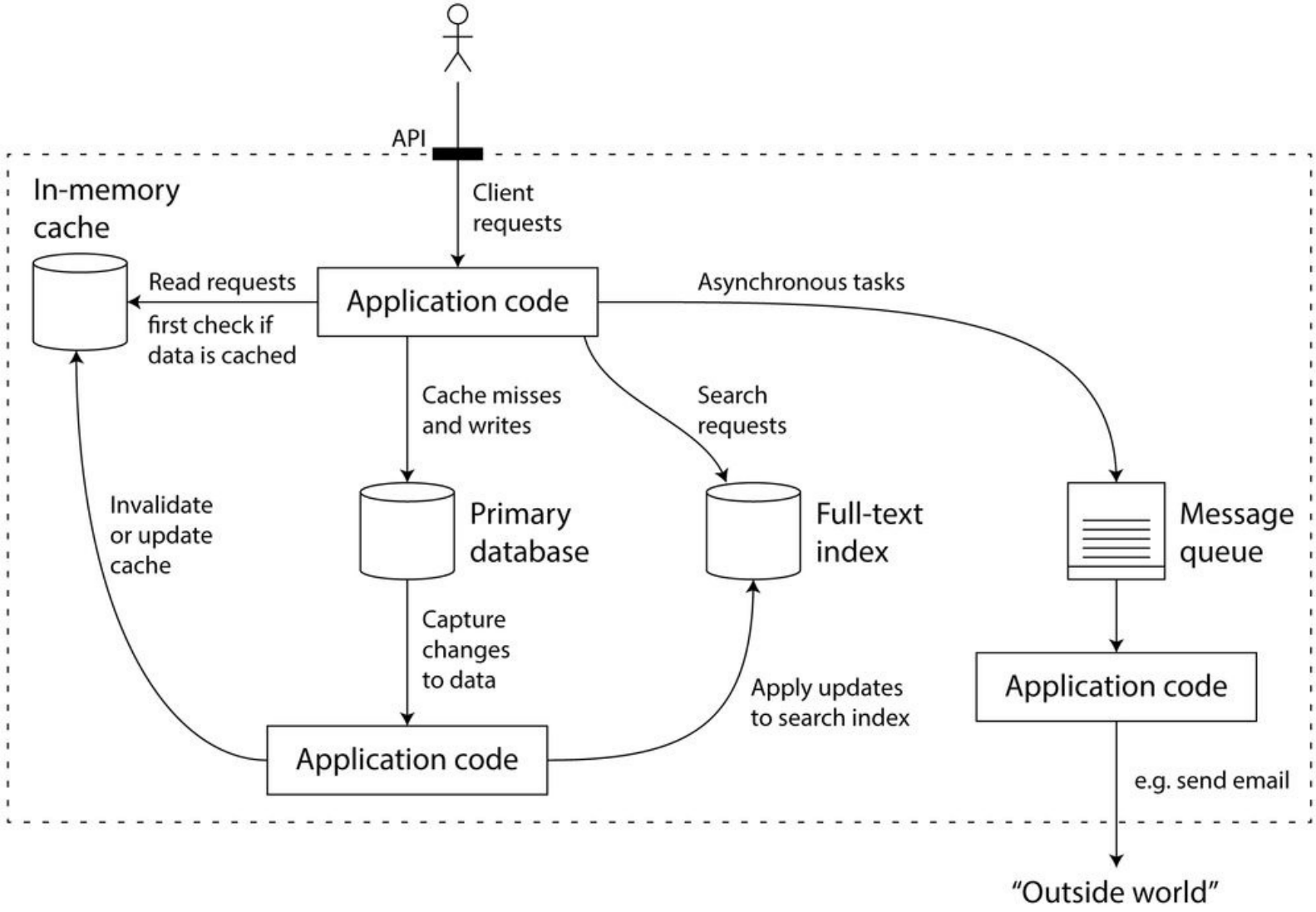
Data systems typically needs to

- Store data so that they, or another application, can find it again later (**databases**).
- Remember the result of an expensive operation, to speed up reads (**caches**).
- Allow users to search data by keyword or filter it in various ways (**search indexes**).
- Send a message to another process, to be handled asynchronously (**message queues**).
- Observe what is happening, and act on events as they occur (**stream processing**).
- Periodically crunch a large amount of accumulated data (**batch processing**).

Thinking about Data Systems

- Increasingly, many applications have wide-ranging requirements
  - Many times, a single tool can no longer meet all of its data processing and storage needs.
- Instead, the work is broken down into tasks that can be performed efficiently on a single tool,
  - the different tools are stitched together using application code.
- For example, we may have an application with:
  - a caching layer (e.g. memcached or similar),
  - a full-text search server (e.g. Elasticsearch or Solr),
  - separated from the main database (e.g. MySQL).

Thinking about Data Systems



Data Systems – some challenges

- How do you ensure that the data remains correct and complete,
  - even when things go wrong internally?
- How do you provide consistently good performance to clients,
  - even when parts of your system are degraded?
- How do you scale to handle an increase in load?
- What does a good API for the service look like?

Data Systems – some requirements

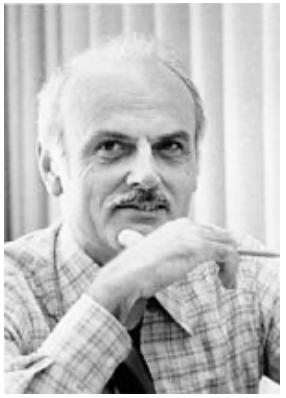
- **Reliability**: The system should continue performing the correct function at the desired performance,
  - even in the face of adversity (hardware or software faults, and even human error).
- **Scalability**: As the system grows (in data volume, traffic volume or complexity), there should be reasonable ways of dealing with that growth.
- **Maintainability**: Over time, many different people should all be able to work on it productively,
  - Engineering and operations, both maintaining current behavior and adapting the system to new use cases.

Database Systems

- A "database" is normally referred as a **set of related data** and its **organization**.
- A "database management system" (**DBMS**) controls the access to this data.
  - Providing functions that allow writing, searching, updating, retrieving, and removing large quantities of information.

Brief History of Database Systems

- Pre-relational era (1970's)
  - Hierarchical (IMS), Network (Codasyl)
  - Many database systems
    - Complex data structures and low-level query language
    - Incompatible, exposing many implementation details
- **Relational DBMSs** (1980s)
  - Edgar F. Codd's relational model in 1970
  - Powerful high-level query language
  - A few major DB systems dominated the market



- Object-Oriented DBMSs (1990s)
  - Motivated by "mismatch" between RDBMS and OO PL
  - Persistent types in C++, Java or Small Talk
  - Issues: Lack of high level QL, no standards, performance

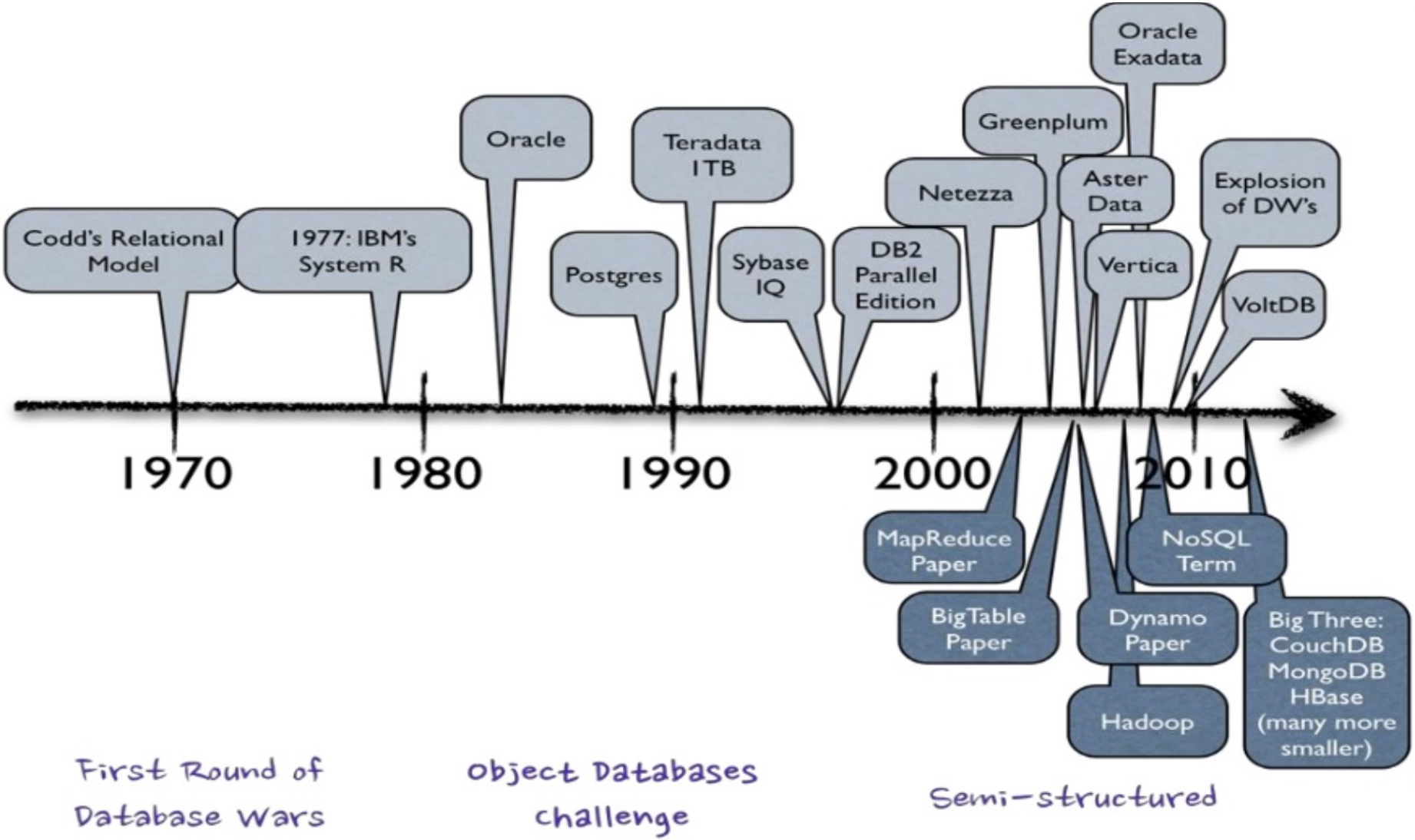
Brief History of Database Systems

- Object-relational DBMS (OR-DBMS) (1990s)
  - Relational DBMS vendors' answer to OO
  - User-defined types, functions (spatial, multimedia) Nested tables
  - SQL: 1999 (2003) standards. Plus performance.
- XML/DBMS (2000s)
  - Web and XML are merging
  - Native support of XML through ORDBMS extension or native XML DBMS
- Data analytics system (DSS) (2000s)
  - **Data warehousing and OLAP**

Brief History of Database Systems

- Data stream management systems (2000s)
  - Continuous query against data streams
- The era of big data (mid 2000-now):
  - **Big data**: datasets that grow so large (terabytes to petabytes) that they become awkward to work with traditional DBMS
  - Parallel DBMSs continue to push the scale of data
  - **MapReduce** dominates on Web data analysis
  - **NoSQL** (not only SQL) is fast growing

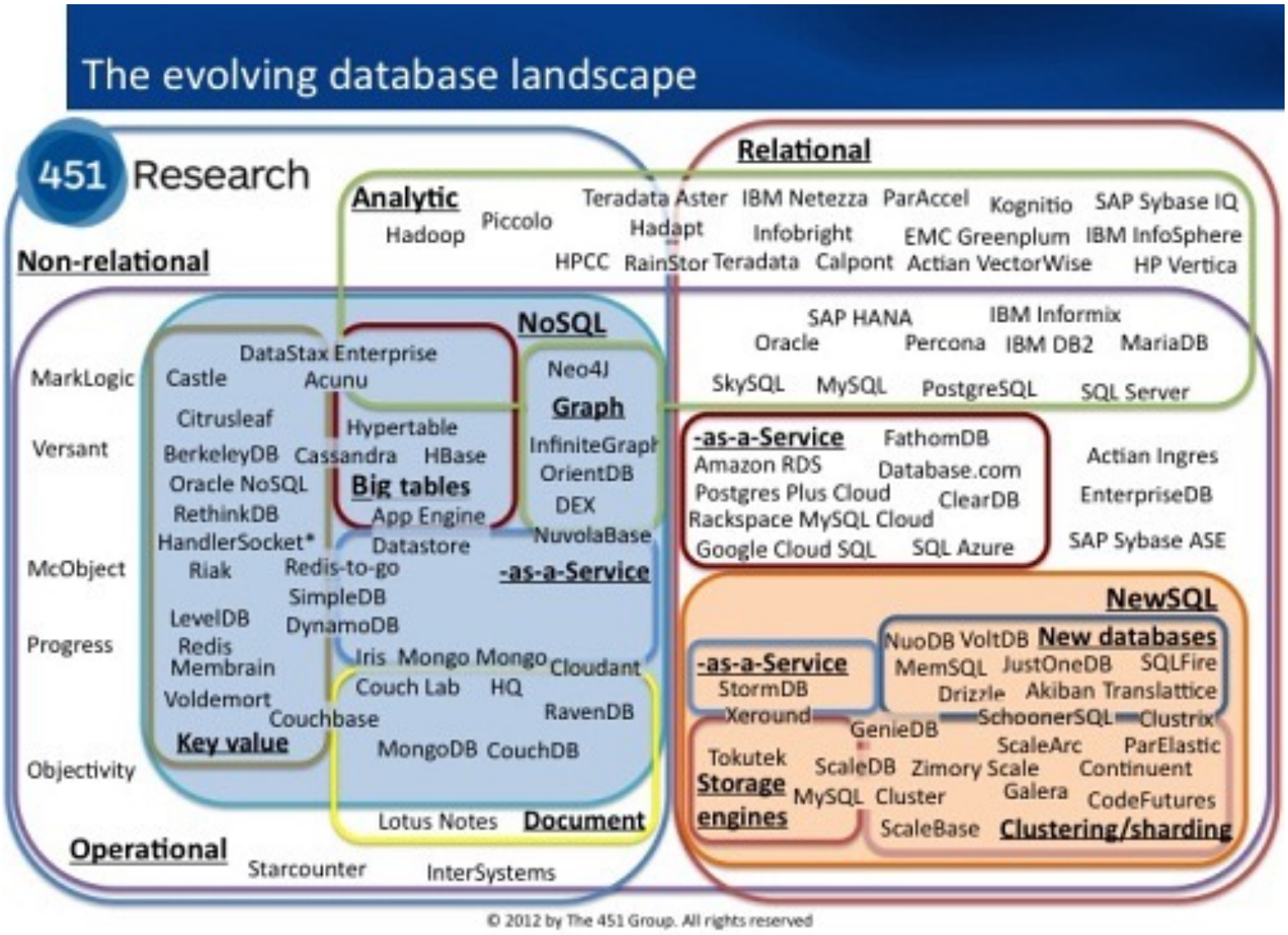
Database Evolution Timeline

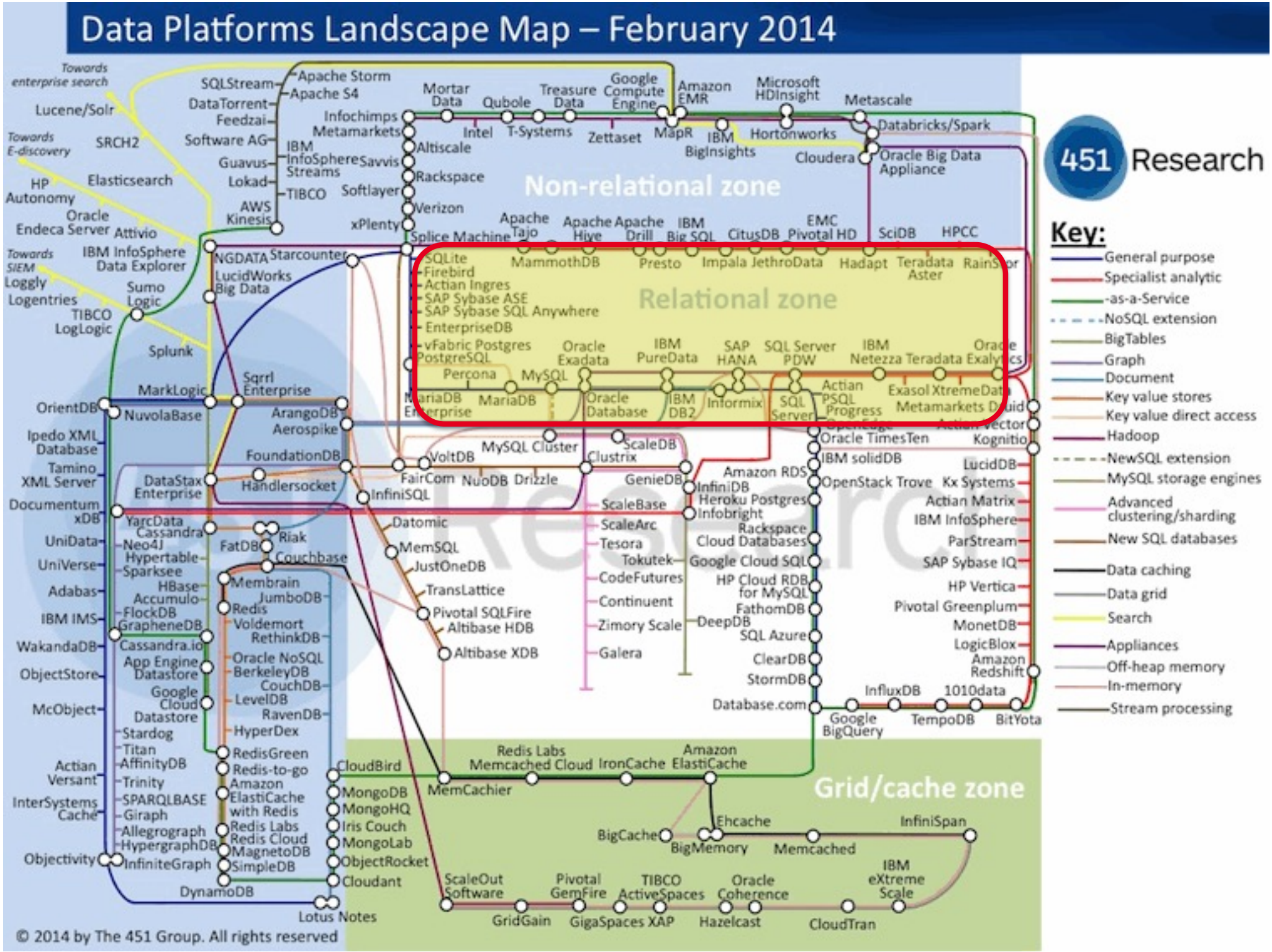


"Big data – an introduction", Peter Morgan

Database Systems Landscape

Database Systems Landscape




Data Platforms Landscape Map – February 2014
451 Research
Towards enterprise search
Towards E-discovery
Towards SIEM
Lucene/Solr
SRCH2
Elasticsearch
HP Autonomy
Oracle Endeca Server
Attivio
IBM InfoSphere Data Explorer
Loggly
Logentries
Sumo Logic
TIBCO LogLogic
Splunk
SQLStream
DataTorrent
Feedzai
Software AG
Guavus
Lokad
AWS Kinesis
Apache Storm
Apache S4
IBM InfoSphere Streams
TIBCO
Infochimps
Metamarkets
Savvis
Softlayer
xPlenty
Mortar Data
Qubole
Treasure Data
Google Compute Engine
Amazon EMR
Microsoft HDInsight
Metascale
Intel
T-Systems
Zettaset
MapR
IBM BigInsights
Hortonworks
Cloudera
Databricks/Spark
Oracle Big Data Appliance
Altiscale
Rackspace
Verizon
Non-relational zone
Apache Tajo
Apache Hive
Apache Drill
IBM Big SQL
CitusDB
EMC Pivotal HD
SciDB
HPCC
Splice Machine
NGDATA
LucidWorks Big Data
Starcounter
SQLite
Firebird
Actian Ingres
SAP Sybase ASE
SAP Sybase SQL Anywhere
EnterpriseDB
vFabric Postgres
PostgreSQL
Relational zone
MammothDB
Presto
Impala
JethroData
Hadapt
Teradata Aster
Oracle Exadata
IBM PureData
SAP HANA
SQL Server PDW
IBM Netezza
Teradata
Percona
MySQL
MariaDB Enterprise
MariaDB
Oracle Database
IBM DB2
Informix
SQL Server
Actian PSQL
Progress
Exasol
Metamarkets
Kognitio
MarkLogic
Sqrrl Enterprise
OrientDB
NuvolaBase
ArangoDB
Aerospike
FoundationDB
Handlersocket
InfiniSQL
VoltDB
FairCom
NuoDB
Drizzle
MySQL Cluster
Clustrix
ScaleDB
GenieDB
ScaleBase
ScaleArc
Tesora
Tokutek
CodeFutures
Continuent
Zimory Scale
Galera
Oracle TimesTen
IBM solidDB
InfiniDB
Heroku Postgres
Infobright
Amazon RDS
Rackspace Cloud Databases
Google Cloud SQL
HP Cloud RDB for MySQL
FathomDB
DeepDB
SQL Azure
ClearDB
StormDB
Database.com
OpenStack Trove
LucidDB
Kx Systems
Actian Matrix
IBM InfoSphere
ParStream
SAP Sybase IQ
HP Vertica
Pivotal Greenplum
MonetDB
LogicBlox
Amazon Redshift
InfluxDB
1010data
Google BigQuery
TempoDB
BitYota
Ipedo XML Database
Tamino XML Server
DataStax Enterprise
Documentum xDB
UniData
UniVerse
Adabas
IBM IMS
WakandaDB
ObjectStore
McObject
Actian Versant
InterSystems Caché
Objectivity
YarcData
Cassandra
Neo4J
Hypertable
Sparksee
HBase
Accumulo
FlockDB
GrapheneDB
Cassandra.io
App Engine Datastore
Google Cloud Datastore
Stardog
Titan
AffinityDB
Trinity
SPARQLBASE
Giraph
Allegrograph
HypergraphDB
InfiniteGraph
Riak
FatDB
Couchbase
Membrain
JumboDB
Redis
Voldemort
RethinkDB
Oracle NoSQL
BerkeleyDB
CouchDB
LevelDB
RavenDB
HyperDex
RedisGreen
Redis-to-go
Amazon ElastiCache with Redis
Redis Labs
Redis Cloud
MagnetoDB
SimpleDB
DynamoDB
Lotus Notes
Datomic
MemSQL
JustOneDB
TransLattice
Pivotal SQLFire
Altibase HDB
Altibase XDB
Redis Labs Memcached Cloud
IronCache
Amazon ElastiCache
CloudBird
MongoDB
MongoHQ
Iris Couch
MongoLab
ObjectRocket
Cloudant
MemCachier
Grid/cache zone
BigCache
Ehcache
BigMemory
Memcached
InfiniSpan
ScaleOut Software
Pivotal GemFire
TIBCO ActiveSpaces
Oracle Coherence
IBM eXtreme Scale
GridGain
GigaSpaces XAP
Hazelcast
CloudTran
Key:
General purpose
Specialist analytic
-as-a-Service
NoSQL extension
BigTables
Graph
Document
Key value stores
Key value direct access
Hadoop
NewSQL extension
MySQL storage engines
Advanced clustering/sharding
New SQL databases
Data caching
Data grid
Search
Appliances
Off-heap memory
In-memory
Stream processing
© 2014 by The 451 Group. All rights reserved

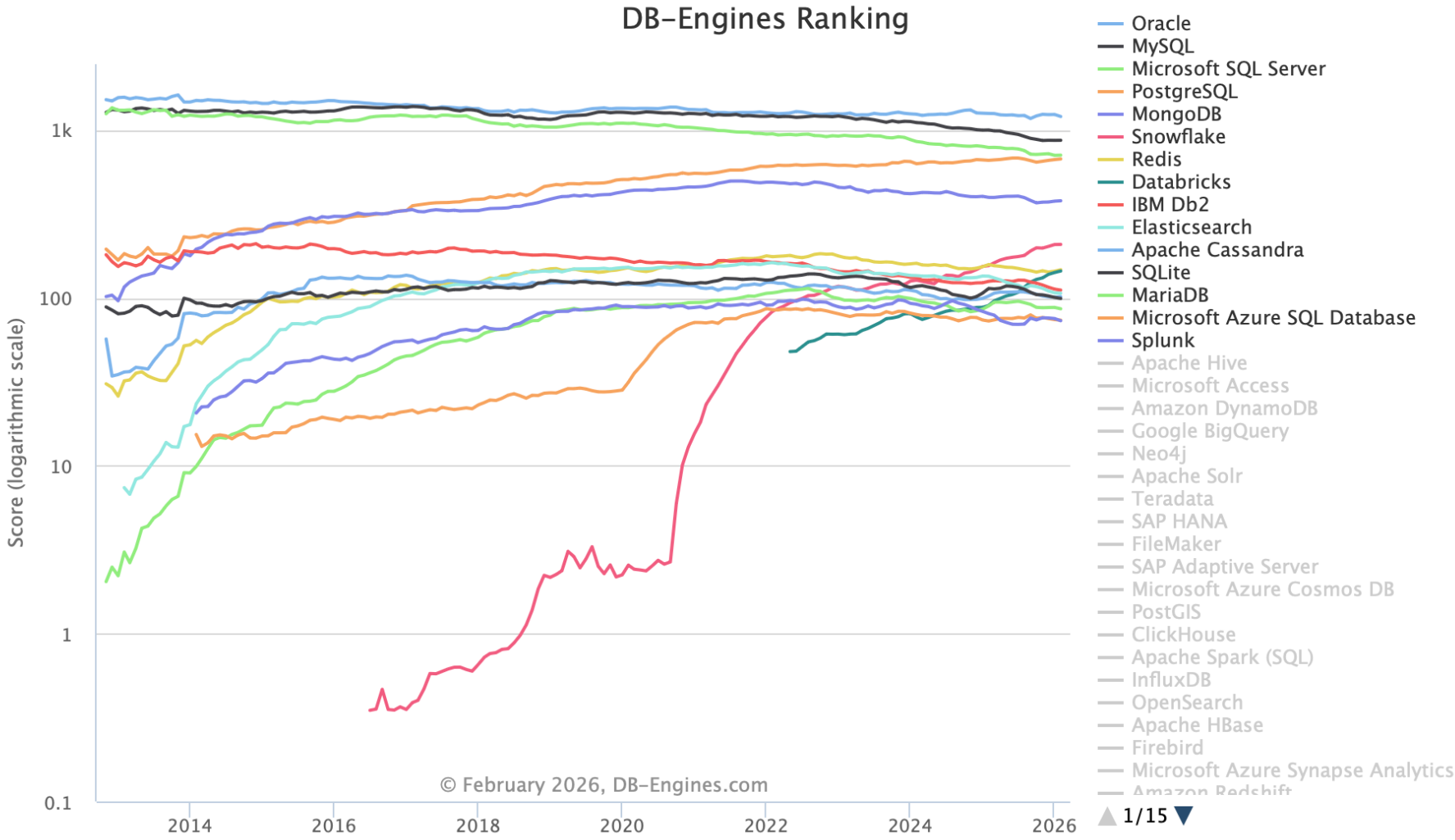
Database Systems Landscape

429 systems in ranking, February 2026

Rank			DBMS	Database Model	Score		
Feb 2026	Jan 2026	Feb 2025			Feb 2026	Jan 2026	Feb 2025
1.	1.	1.	Oracle	Relational, Multi-model	1203.51	-33.82	-51.31
2.	2.	2.	MySQL	Relational, Multi-model	868.22	+0.70	-131.77
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	708.14	+1.88	-78.73
4.	4.	4.	PostgreSQL	Relational, Multi-model	672.03	+5.77	+12.42
5.	5.	5.	MongoDB	Multi-model	378.73	+1.99	-17.90
6.	6.	↑ 7.	Snowflake	Relational	208.14	+0.34	+52.56
7.	7.	↓ 6.	Redis	Key-value, Multi-model	147.04	+2.88	-10.87
8.	8.	↑ 13.	Databricks	Multi-model	144.51	+2.97	+54.48
9.	9.	9.	IBM Db2	Relational, Multi-model	111.22	-1.50	-14.21
10.	10.	↓ 8.	Elasticsearch	Multi-model	106.46	-0.69	-28.17
11.	11.	11.	Apache Cassandra	Wide column, Multi-model	101.60	+0.76	-0.98
12.	12.	↓ 10.	SQLite	Relational	99.19	-1.42	-14.63
13.	13.	↑ 14.	MariaDB	Relational, Multi-model	86.09	-1.64	-3.42
14.	↑ 15.	↑ 17.	Microsoft Azure SQL Database	Relational, Multi-model	73.66	-0.53	+0.90
15.	↓ 14.	15.	Splunk	Search engine	73.05	-2.25	-7.51
16.	16.	↑ 18.	Apache Hive	Relational	73.00	-0.48	+10.51
17.	17.	↓ 12.	Microsoft Access	Relational	69.47	-2.18	-27.07
18.	18.	↓ 16.	Amazon DynamoDB	Multi-model	67.87	-1.92	-7.71
19.	19.	19.	Google BigQuery	Relational	59.58	-4.02	+4.69
20.	20.	20.	Neo4j	Graph	49.26	-0.89	+3.92

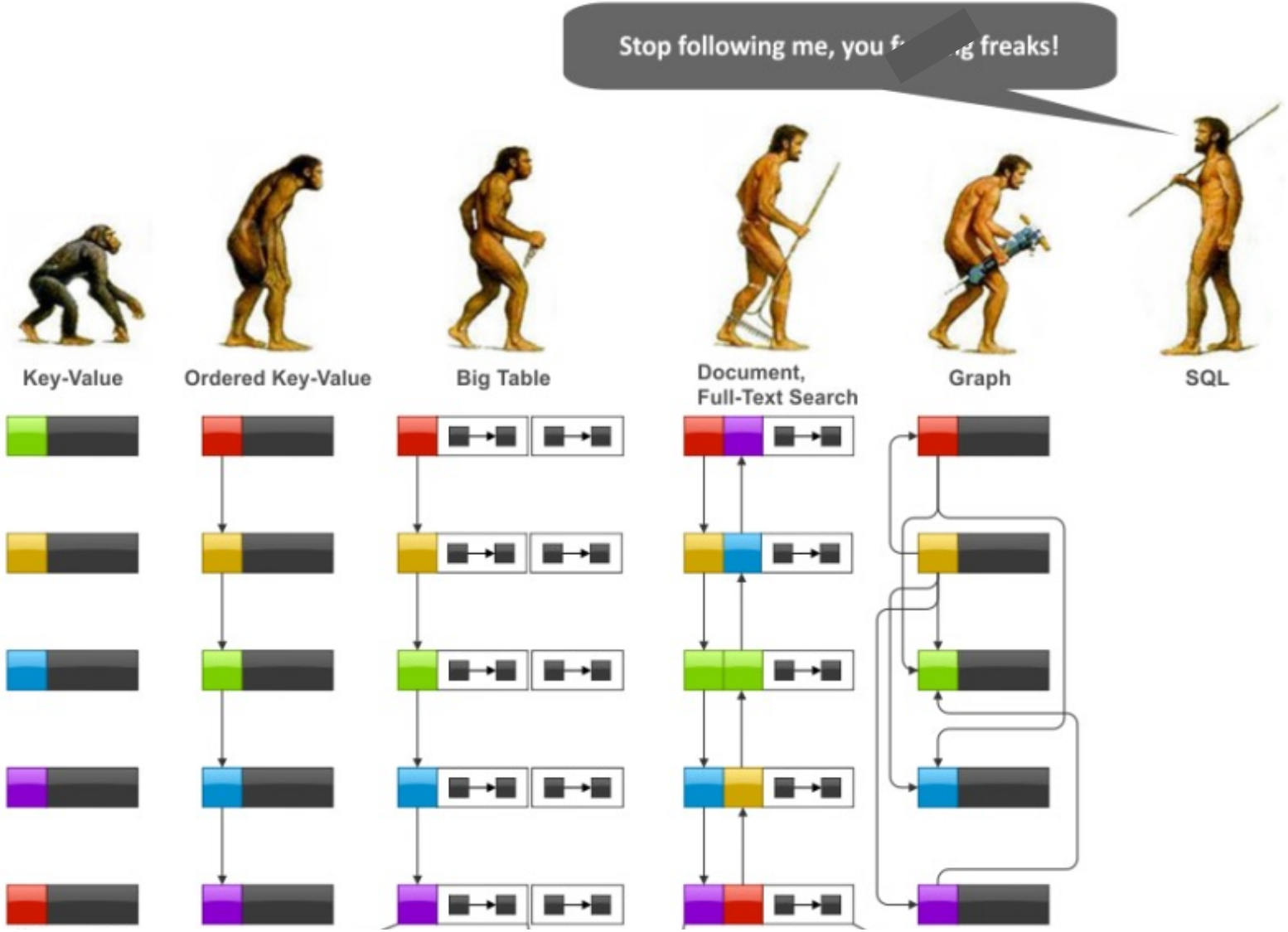
https://db-engines.com/en/ranking

Database Systems Landscape



https://db-engines.com/en/ranking_trend

Database Systems Landscape



https://highlyscalable.wordpress.com/2012/03/01/nosql-data-modeling-techniques/

Resources

- Martin Kleppmann, ***Designing Data-Intensive Applications***, O'Reilly Media, Inc., 2017.
- Pramod J Sadalage and Martin Fowler, ***NoSQL Distilled*** Addison-Wesley, 2012.
- Eric Redmond, Jim R. Wilson. ***Seven databases in seven weeks***, Pragmatic Bookshelf, 2012.
- Hector Garcia-Molina, Jeffrey D. Ullman, Jennifer Widom, ***Database systems: the complete book*** *(2nd Ed.)*, Pearson Education, 2009.